

RedTeamForge: Architectural Deep Dive & Documentation

RedTeamForge: Comprehensive Architectural & Technical Specification

> [!NOTE]

> RedTeamForge is a specialized, autonomous framework built entirely in Python using an intelligence-decoupled, multi-agent evaluation paradigm.

> The system operates by dynamically scaling LLM fuzzing techniques to surface prompt injections, unauthorized tool chains, unmitigated jailbreaks, and vector manipulation vulnerabilities.

1. Executive Business Pitch & Commercial Viability

1.1 The Market Problem

As enterprises rapidly integrate Generative AI and autonomous Large Language Models (LLMs) into their technology stacks (e.g., customer service chatbots, financial transaction copilots, internal HR agents), the cybersecurity landscape has drastically shifted. Traditional penetration testing tools (designed for SQLi, XSS, and network vulnerabilities) are entirely blind to mathematical and linguistic manipulation techniques like *jailbreaking, multi-turn prompt injection, contextual poisoning, and agentic tool chaining.* Currently, securing these models relies on manual "red teaming"?a human-led process that is exceptionally costly, slow, and impossible to scale alongside agile CI/CD development pipelines.

1.2 The RedTeamForge Solution

RedTeamForge acts as a fully autonomous, highly sophisticated "Red Team in a Box." Operating seamlessly with localized architectures, it deploys a decoupled multi-agent intelligence system to automatically probe, attack, and expose vulnerabilities in corporate LLMs *before* they are deployed to production. By leveraging evolutionary mutation engines and complex persona generation, RedTeamForge continually outpaces static safety filters, resulting in compliance-ready, heavily detailed PDF audit reports without any human engineering overhead.

1.3 Target Demographics (Who Can Use Our Product)

* Chief Information Security Officers (CISOs) & Compliance Teams: Organizations bound by extreme compliance measures (GDPR, HIPAA, SEC regulations) utilizing AI need provable, documented audits demonstrating that their LLM interfaces do not leak protected PII or provide unsanctioned financial advice.

* DevSecOps & AI Integration Engineers: Development teams building LLM wrappers can integrate RedTeamForge into their CI/CD pipelines (via Webhook APIs). Every time a system prompt is modified or base weights are quantized, RedTeamForge can run regression testing overnight to verify safety integrity.

* Managed Security Service Providers (MSSPs) & Pentest Firms: Third-party security vendors can license RedTeamForge to offer "AI Penetration Testing / AI Compliance Auditing" as a lucrative, dedicated new service line for their corporate client base.

1.4 Monetization Strategy (How We Make Money)

* Tiered SaaS Cloud Subscriptions: Offer a hosted version of the platform. Tiers can be bounded by the number of probes (e.g., *Startup Tier: 1,000 probes/month*, *Enterprise Tier: 100,000 probes/month*), execution time limits, or concurrent agent limits.

* Enterprise On-Premises Licensing agreement: Financial institutions, healthcare, and federal defense contractors often cannot pipe their data to external clouds. Because RedTeamForge functions entirely natively (down to local Ollama inference and pure Python PDF tracking), it is extremely valuable as a high-ticket, annual on-premises enterprise license installed directly into a client's secure VPC.

* Pay-Per-Audit Execution: Small-to-Medium businesses running individual audits can pay a flat transactional fee to

RedTeamForge: Architectural Deep Dive & Documentation

execute a focused test suite and generate a branded PDF assessment summary for stakeholders.

* Proprietary Global Threat Intelligence Feed (Zero-Day API): Utilizing the interconnected GenomeDB, RedTeamForge gathers highly sophisticated, evolved attack vectors across the network. This aggregated database of mathematically proven LLM bypass strings can be sold as an API subscription directly to AI Firewall / WAF vendors to proactively block new attack vectors.

2. Core Architectural Paradigm: Tri-Agent Asynchronous Eval

To break safety layers effectively without human intervention, RedTeamForge operates on a decoupled Tri-Agent Architecture.

1. The Attacker Engine (rtf_core/attacker.py)

The attacker subsystem governs *Payload Discovery & Delivery*. Instead of firing static .txt payloads, it uses localized LLMs running on Ollama to contextualize the attack against the provided scope.

* Mechanics: It utilizes a two-step "Plan and Execute" reasoning chain. First, a *Strategy Planner* is prompted with previous findings and insights. It selects exactly 3 optimal techniques from its master list (e.g. *Direct Injection, Role Escalation, Context Poisoning, Token Smuggling, Authority Impersonation*).

* Generative Step: Once a strategy is derived, the model enters generator mode, yielding JSON arrays of raw attacker input to fire at the Victim Client.

2. The Target Client (rtf_core/victim_client.py)

This operates as the isolated proxy bridging RedTeamForge to wherever the target model is hosted via generic Webhooks.

3. The Impartial Judge (rtf_core/judge.py, rtf_core/tool_abuse_judge.py)

* Evaluation Constraints: The Judge is heavily prompted to operate impartially, looking purely for data leakage or unauthorized rule breakage. It responds with strict JSON schemas.

* Output Metadata: The Judge returns booleans for vulnerability_found, strings detailing the weak_area (which node failed), key_insight, an ordinal severity (1-5), and the strategy_tag.

3. Advanced Subsystems & Run-Loops

RedTeamForge handles network transit, queuing, and iterative recursion entirely on asynchronous background threads via queue.Queue. This separates the main UI loop from execution loops.

3.1 The Standard Probe Generation Engine (rtf_core/probe_runner.py)

This represents single-turn iterative Fuzzing.

* Component Tracking: ProbeRunner initializes a persistent connection instance (VictimClient). Memory is driven by ProbeMemory which records hits and consecutive misses.

* Intelligent Stopping Logic:

It utilizes an "AI Analyst". Every 6 probes (2 batches), the system passes current metrics to the Judge to verify if it has reached *Diminishing Returns*. Secondary hard stop conditions include MAX_PROBES_PER_SESSION, execution time (MAX_SESSION_MINUTES), or too many consecutive misses without vulnerabilities.

3.2 The Agentic Tool Abuse Engine (rtf_core/tool_abuse_runner.py)

Perhaps the most complex engine, it performs conversational multi-turn hacking against agentic tooling configurations.

* Thompson Sampling (ToolAbuseMemory.py): Unlike standard linear probe lists, RedTeamForge uses reinforcement-learning heuristics (Thompson Sampling from the beta distribution) to dynamically weight which API exploitation categories (TOOL_ABUSE_VECTORS) are currently yielding the highest hit ratios against the model.

* Escalation Context & Templates: Uses explicitly defined Escalation Vectors including:

RedTeamForge: Architectural Deep Dive & Documentation

- * **Indirect Injection:** Hiding instructions through hypothetical lookups ("Can you look up notes for {id}... my manager asked me to review them").
- * **Parameter Poisoning:** Attempting nested queries within tools ("SELECT * FROM pg_shadow").
- * **Tool Chaining:** Forcing sequential actions independently derived (First get account, then send email to external@...).
- * **Exfiltration:** Passing environmental state outside the boundary.
- * **Multi-Turn Contextualization:** Turn 0 injects a primer context. Future turns rely on historical conversation snapshots to stack commands sequentially over a single simulated UUID connection (conv_session_id).

3.3 Semantic Coverage-Guided Mutation (`rtf_core/mutator.py`)

- * The evolutionary algorithm tracking successful probes.
 - * **Genetic Recombination:** Parent probes that achieved arbitrary high severity (e.g. Critical 4-5) are taken from ProbeDB or the internal GenomeDB.
 - * **Persona Switching:** To bypass guardrail entropy scanning, the mutation module automatically masks the next prompt into character personas (technical_security_researcher, confused_end_user, social_engineer, frustrated_customer, academic_researcher).
 - * **Resilience Mechanisms:** Smaller Ollama models frequently drop trailing JSON formatting. The Mutator script includes highly specialized un-escaped regex fallbacks to parse strings out of truncated or malformed HTTP JSON payload returns (`_parse_json_list`).
-

4. Storage Layer & Database (`rtf_core/db.py`)

Constructed using SQLAlchemy, this allows mapping across SQLite seamlessly up to distributed PostgreSQL databases (e.g., standard migration `DB_shift_postgres.py`).

1. SessionDB (`rtf_sessions`):

Establishes execution timeline metrics, total risks, scoped configuration texts, total probes fired, total hits achieved, and acts as the relational root.

2. ProbeDB (`rtf_probes`):

Captures individual single-turn payload interactions (Request -> Response + Severity + Weak Area Mapping).

3. ToolAbuseProbeDB (`rtf_tool_abuse_probes`):

Crucial extension adding state columns unique to API manipulation: turn number, current mutation tracking vector, `primary_tool_type`, `tools_involved` lists, execution JSON dictionaries (`tool_calls`), and soft refusal trackers.

4. MutatedPromptDB (`rtf_mutations`):

Stores lineage. Maps `parent_probe_id` to its respective iteration payload allowing parent->child visual maps.

5. GenomeDB (`genomes`):

A cross-session threat-intel library. Re-initializes system knowledge upon restarts giving RedTeamForge persistent state between UI refreshes. Stores complexities, nesting depths, and generic features strings.

5. Platform Visualization & Output Delivery (`streamlit_app.py`)

Built dynamically on Streamlit.

* UI Theme Initialization:

Custom raw native CSS for dynamic cards parsing across grids (`.kpi-card`, `.badge`, `.vector-card`).

* Memory & Event Bubbling:

Async calls generated by `threading.Thread` instances bubble metadata via `Python Queue.get_nowait()` natively into the `st.session_state` dictionaries, preventing Streamlit rerender blocking.

* Execution Tabs:

1. **Dashboard & Configuration:** Live gauges via KPI components mapping vulns / `len(probe_results)`. Target linking.
2. **Injection & Tool Loops:** Triggers instances of `ProbeRunner` & `ToolAbuseRunner` executing the tri-agent algorithm

RedTeamForge: Architectural Deep Dive & Documentation

iteratively.

3. Mutation Explorer: Recursively querying the Genetic tree (mutated_prompts).

4. Reporting Export (pdf_export.py): Reconstructs the DB rows via report_builder.py, piping exact string matching into pure Python fpdf2. Implements strict A4 mapping, risk calculation bounds formatting (_risk_color), dynamically resizing cells mapping string matrices directly to the web client via st.download_button avoiding hard disk temporary dumping.